TESTING AND DEPLOYMENT | CONCEPTS & DEMO

Short speaking guide based on the implemented local application

Shreshth Sharma - 261010286 | Testing and Deployment tools
Project: NoteSphere - Smart Student Notes Sharing Platform

Your 45-60 second opening

"I am responsible for testing and local delivery of NoteSphere. We use the built-in Node.js test runner through tsx for unit tests, TypeScript for static checks, and an integration script that starts an isolated Next.js server with a separate database. The recorded results show four unit tests and seventeen grouped integration checks passing. These cover both successful workflows and denied access. For demonstration, a Windows command file starts localhost and opens the browser. The current deliverable is local; I will not claim a verified Vercel deployment or email delivery."

Tools and commands to explain

TOOL / COMMAND	WHAT IT DOES
npm test	Runs tsx --test tests/*.test.ts using node:test and node:assert/strict. Tests password hashing, access rules, validation and origin checks. Vitest is not used.
npm run typecheck	Runs tsc --noEmit to detect TypeScript errors. Passing this does not prove every runtime behavior.
npm run test:flows	Runs scripts/test-flows.ts against an isolated local development server on port 3101 and a separate PGlite directory. It exercises real HTTP/database interactions.
Open-NoteSphere.cmd / npm run local	Launches local Next.js via scripts/start-local.mjs, waits for readiness and opens the browser. Keep the command window open; Ctrl+C stops it.
npm run build / npm start	Commands for a production build and server. They exist in the project; the latest role tests use development mode, not a verified production deployment.

A short live test sequence

1. Sign in as admin and publish a valid PDF.
2. Sign in as one demo student and show the new notification.
3. Search/open the PDF, bookmark it and add it to a collection.
4. Review it, sign out/in, and verify the library persists.
5. Show student upload/admin access is denied and a private draft is unavailable.

Evidence: docs/TEST-RESULTS.md records 17 grouped role integration checks. Four unit tests and TypeScript checks passed in the project verification. These are recorded results, not a substitute for rehearsing the current machine.

TESTING AND DEPLOYMENT | QUESTIONS & CODE MAP

Use these answers after rehearsing the actual code and demonstration

What is the difference between unit and integration testing?

Unit tests check small rules in isolation, such as password verification or input validation. Integration tests send real requests through the server and database, such as upload, notification delivery and persistent bookmarks.

Which negative cases were tested?

Guests cannot retrieve PDFs; students cannot upload or call admin APIs; forged roles are ignored; invalid/reused admin invitations fail; disguised/non-PDF documents are rejected; private drafts remain hidden; cross-site mutation requests fail.

How do you avoid changing real demo data during tests?

The flow script uses an isolated database and separate server/cache paths. Test passwords are randomized inside that test database. The script does not need a cloud project or the live user database.

Does localhost need internet?

With Node.js and dependencies already installed, the local launcher works offline using embedded PostgreSQL and disk storage. The initial npm ci installation may require internet. SMTP email and cloud integrations are disabled in this local launcher.

What if the server does not open?

Read .data/local-launch.log. Verify Node.js and installed dependencies, keep the launcher window open, and use the URL it prints. The launcher searches for a free port from 3000 to 3019 and can reopen its existing instance.

Has deployment or browser rendering been verified?

The recorded integration suite verifies HTTP responses and database behavior, including returned PDF bytes. It does not establish visual browser rendering, real SMTP delivery or a successful cloud deployment. Vercel configuration is present, but the current demonstration is localhost.

Files to show

tests/security.test.ts + scripts/test-flows.ts
The actual unit assertions and complete role workflow checks.

scripts/start-local.mjs + Start-NoteSphere.cmd
Offline local startup, free-port detection, readiness checks and startup logs.

Honest closing line: "We tested the main role and persistence workflows locally. Cloud hosting and email still need separate configuration and verification."